

PCE Complete Run Guide — Backend First, Then Frontend

Prerequisites: Docker Desktop (or Docker Engine + Compose), Git, Flutter SDK, and an Android emulator or physical device.

PART 1: BACKEND (price_comparision_engine)

Step 1 — Clone the repo

```
git clone https://github.com/mukro/price_comparision_engine.git
cd price_comparision_engine
```

Step 2 — Unzip the improvements package (if not already extracted)

```
unzip -o price_engine_v2_improvements.zip -d ./
```

Step 3 — Create environment file

```
cp .env.example .env
```

Edit `.env` and fill in real values. Here is a **minimum viable** `.env` for local development:

```
# --- Database ---
POSTGRES_USER=price_engine
POSTGRES_PASSWORD=your_strong_password_here
POSTGRES_HOST=db
POSTGRES_PORT=5432
POSTGRES_DB=price_comparison

# --- Redis ---
REDIS_URL=redis://:your_redis_password_here@redis:6379/0
REDIS_PASSWORD=your_redis_password_here

# --- JWT ---
JWT_SECRET_KEY=generate_a_32_char_random_string_here
JWT_ALGORITHM=HS256
JWT_EXPIRE_MINUTES=60

# --- Admin ---
ADMIN_EMAIL=admin@yourdomain.com
ADMIN_PASSWORD_HASH=$2b$12$EixZaYVK1fsbw1ZfbX3OXePaWxn96p36WQoeG6Lruj3vjPGga31lW

# --- Email (SendGrid) ---
SENDGRID_API_KEY=SG.xxxxxx
FROM_EMAIL=alerts@yourdomain.com

# --- CORS ---
ALLOWED_ORIGINS=http://localhost:3000,http://10.0.2.2:8000

# --- Flower ---
```

```
FLOWER_USER=admin
FLOWER_PASSWORD=flower_password_here

# --- Grafana ---
GRAFANA_ADMIN_PASSWORD=grafana_password_here

# --- Agent Flag (LEAVE FALSE until you have traction) ---
AGENTS_ENABLED=false
OPENAI_API_KEY=sk-xxxxxxx
ANTHROPIC_API_KEY=sk-ant-xxxxxxx
```

Generate a real bcrypt hash for admin password:

```
python3 -c "from passlib.hash import bcrypt; print(bcrypt.hash('your_password'))"
```

Step 4 — Build and start all services

```
docker compose up --build -d
```

This starts:

- PostgreSQL + pgVector (port 5432, localhost-only)
- Redis (internal-only, no host port)
- FastAPI (port 8000)
- Celery Worker (default queue)
- Celery Worker (high_priority queue)
- Celery Beat (scheduler)
- Flower (port 5555)
- Prometheus (port 9090)
- Grafana (port 3000)

The agent orchestrator is NOT started (it uses the agents profile and AGENTS_ENABLED=false).

Step 5 — Run database migrations

```
docker compose exec -T db psql -U price_engine -d price_comparison < scripts/migrate_v1_to_v2.sql
```

If you also have monetization schema:

```
docker compose exec -T db psql -U price_engine -d price_comparison < monetization_foundation_v1.sql
```

Step 6 — Verify everything is healthy

```
# 1. API health
curl http://localhost:8000/health
# Expected: {"status":"healthy","version":"2.0.0",...}

# 2. API docs
curl http://localhost:8000/docs
```

```
# Should return Swagger HTML

# 3. Agent health (should show standby)
curl http://localhost:8000/health/agents
# Expected: {"agents_enabled":false,"status":"standby",...}

# 4. Flower dashboard
open http://localhost:5555
# Login with FLOWER_USER / FLOWER_PASSWORD from .env

# 5. Grafana dashboard
open http://localhost:3000
# Login with admin / GRAFANA_ADMIN_PASSWORD

# 6. Check all containers are up
docker compose ps
```

Step 7 — Get an admin token (for admin/merchant APIs)

```
curl -X POST http://localhost:8000/api/v1/admin/auth/login -H "Content-Type: application/json" -d
  ↪ '{"email":"admin@yourdomain.com","password":"your_password"}'
```

Save the `access_token` returned. Use it as `Authorization: Bearer <token>` on all `/api/v1/admin/*` and `/api/v1/merchant/*` endpoints.

Step 8 — (Optional) Expose via Cloudflare Tunnel

If you want the backend accessible from the Flutter app on a real device or from the internet:

```
# Install cloudflared
wget https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-linux-amd64.deb
sudo dpkg -i cloudflared-linux-amd64.deb

# Login (one-time)
cloudflared tunnel login

# Create a tunnel
cloudflared tunnel create pce-api

# Route traffic
cloudflared tunnel route dns pce-api api.yourdomain.com

# Run the tunnel
cloudflared tunnel run pce-api
```

Then update your `.env` `ALLOWED_ORIGINS` to include `https://api.yourdomain.com` and restart:

```
docker compose restart api
```

PART 2: FRONTEND (PCE_App — Flutter)

Step 1 — Clone the repo

```
git clone https://github.com/mukro/PCE_App.git
cd PCE_App
```

Step 2 — Install Flutter dependencies

```
flutter pub get
```

Step 3 — Update the API base URL

Open lib/services/api_service.dart and change line 18:

For Android emulator (backend on same machine):

```
static const String baseUrl = 'http://10.0.2.2:8000/api/v1';
```

For iOS simulator (backend on same machine):

```
static const String baseUrl = 'http://localhost:8000/api/v1';
```

For real Android device (same WiFi):

```
static const String baseUrl = 'http://192.168.1.XX:8000/api/v1'; // your machine IP
```

For production (Cloudflare Tunnel):

```
static const String baseUrl = 'https://api.yourdomain.com/api/v1';
```

Also update lib/services/pce_api_service.dart with the same URL.

Step 4 — Run the app

On Android emulator:

```
flutter emulators --launch <emulator_id>
flutter run
```

On iOS simulator:

```
open -a Simulator
flutter run
```

On a real device (connected via USB with debugging enabled):

```
flutter devices      # find your device ID
flutter run -d <device_id>
```

Build APK for testing on Android:

```
flutter build apk --release
# APK will be at: build/app/outputs/flutter-apk/app-release.apk
```

Step 5 — Verify app connectivity

1. Open the app on your emulator/device
2. Go to the **Search** tab
3. Search for a product (e.g., “iPhone 15”)
4. You should see results from your backend API
5. Check the backend logs to confirm requests are hitting the API:

```
docker compose logs -f api
```

PART 3: Daily Development Workflow

Start everything (after a reboot)

```
cd ~/price_comparision_engine
docker compose up -d
curl http://localhost:8000/health
```

Stop everything

```
docker compose down
```

View logs

```
docker compose logs -f api           # API logs
docker compose logs -f celery_worker # Worker logs
docker compose logs -f celery_beat   # Scheduler logs
docker compose logs -f db            # Database logs
```

Restart a single service

```
docker compose restart api
docker compose restart celery_worker
```

Rebuild after code changes

```
docker compose up -d --build api
```

Database access (psql)

```
docker compose exec db psql -U price_engine -d price_comparison
```

Redis CLI

```
docker compose exec redis redis-cli -a your_redis_password
```

PART 4: Enabling Agents Later (When Ready)

When you have user traction and want to activate agents:

```
cd ~/price_comparision_engine

# 1. Edit .env
# AGENTS_ENABLED=true

# 2. Restart core services
docker compose up -d

# 3. Start agent container (uses the agents profile)
docker compose --profile agents up -d

# 4. Verify
curl http://localhost:8000/health/agents
# Expected: {"agents_enabled": true, "status": "active", ...}

# 5. Monitor in Flower
open http://localhost:5555
```

Troubleshooting

Problem	Cause	Fix
Connection refused from Flutter	Wrong baseUrl	Use 10.0.2.2 for emulator, LAN IP for real device
401 Unauthorized on admin endpoints	Missing Bearer token	Login first, attach Authorization: Bearer <token> header
CORS error in Flutter	Backend not allowing origin	Add your device/emulator origin to ALLOWED_ORIGINS in .env
Database migrations fail	Schema already exists	Run DROP SCHEMA public CASCADE; CREATE SCHEMA public; then re-run migrations
Celery tasks not running	Beat not scheduling	Check docker compose logs -f celery_beat
Agent container starts even when disabled	Forgot --profile agents	docker compose down then docker compose up -d (without -profile)

Port Map (Local Development)

Service	URL	Purpose
FastAPI	http://localhost:8000	Main API
Swagger Docs	http://localhost:8000/docs	Interactive API docs
Flower	http://localhost:5555	Celery monitoring
Grafana	http://localhost:3000	Metrics dashboards
Prometheus	http://localhost:9090	Metrics collection
PostgreSQL	localhost:5432	Database (localhost-only)